

โครงงาน

Master of Reins

จัดทำโดย

6804062612161 นางสาวปิยดา บุตรศรีภูมิ

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2568

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

การพัฒนาเกมเป็นงานที่รวมองค์ความรู้หลายด้านไว้ด้วยกัน ทั้งการออกแบบประสบการณ์ผู้เล่น การเขียนโปรแกรมเชิงวัตถุ การจัดการข้อมูลแผนที่ การคำนวณการชน (Collision) การออกแบบส่วนติดต่อผู้ใช้ และการทดสอบระบบภายใต้เงื่อนไขการใช้งานจริง โครงการ “Master of Reins” จึงถูกพัฒนาขึ้นในลักษณะเกมแข่งม้าแบบสองมิติที่ผู้เล่นวิ่งจากซ้ายไปขวา แข่งกับบอทห้าตัว ผ่านจุดตรวจและเส้นชัย โดยมีระบบพลัง stamina การเร่งความเร็ว ไอเทม การต่อสู้ และเส้นทางทางเลือกที่ทำให้การแข่งขันไม่ได้ขึ้นอยู่กับความเร็วเพียงอย่างเดียว

เกมนี้พัฒนาด้วยภาษา Java และใช้ Java Swing/AWT สำหรับหน้าจอและกราฟิกสองมิติ โดยไม่ใช้ game engine สำเร็จรูป ผู้พัฒนาต้องออกแบบวงจรเกม (Game Loop) ระบบโหลดแผนที่จาก Tiled ระบบชนแบบ swept collision ระบบ AI ของบอท และระบบแสดงผลเองทั้งหมด จึงเป็นโครงการที่แสดงให้เห็นการนำหลักการเขียนโปรแกรมเชิงวัตถุและอัลกอริทึมไปใช้กับงานที่มีปฏิสัมพันธ์แบบทันที (Real-time Interactive Application)

1.2 ประเภทของโครงการ

โครงการนี้จัดเป็นเกมคอมพิวเตอร์ประเภท Desktop Game หรือ Standalone Application ที่ทำงานบนเครื่องคอมพิวเตอร์ส่วนบุคคล ใช้รูปแบบการเล่นแบบ Side-scrolling Racing/Action ผสมกับการแข่งขันกับ AI โดยผู้เล่นหนึ่งคนแข่งขันกับบอทห้าตัวบนแผนที่เดียวที่มีจุดเริ่มต้น จุดตรวจสองจุด และเส้นชัย

ลักษณะสำคัญของโครงการมีดังนี้

- เป็นเกมสองมิติที่ใช้มุมมอง Top-Down และเลื่อนฉากตามผู้เล่นจากซ้ายไปขวา
- เป็นเกมผู้เล่นคนเดียว (Single-player) ที่มีบอทควบคุมด้วยคอมพิวเตอร์ ห้าตัว
- เป็นเกมแข่งความเร็วที่มีระบบ stamina การเร่งความเร็ว และการจัดการทรัพยากรระหว่างการแข่งขัน
- มีระบบต่อสู้ การป้องกัน โล่ และไอเทมเสริม ทำให้การแข่งขันมีมิติของการตัดสินใจมากกว่าการกดวิ่งอย่างเดียว
- เป็นโครงการเชิงการศึกษา (Educational Project) ที่มุ่งแสดงการประยุกต์ใช้ Java, OOP, อัลกอริทึม และการออกแบบส่วนติดต่อผู้ใช้

โปรแกรมไม่ได้อยู่ในรูปแบบ Java Applet และไม่ได้ทำงานผ่านเว็บเบราว์เซอร์ แต่เป็นแอปพลิเคชัน Java SE ที่เปิดหน้าต่างด้วย JFrame และเรียกใช้ผ่านไฟล์ JAR หรือคำสั่ง java -jar

1.3 ประโยชน์

ประโยชน์ของโครงการสามารถแบ่งได้เป็นสามด้าน คือ ประโยชน์ต่อผู้พัฒนา ประโยชน์ต่อผู้เล่น และประโยชน์ในเชิงการศึกษา

ประโยชน์ต่อผู้พัฒนา

- ได้ฝึกออกแบบสถาปัตยกรรมโปรแกรมโดยแบ่งเป็นแพ็กเกจ app, world, racer, collision, items, combat และ bot อย่างชัดเจน
- ได้ใช้หลักการเขียนโปรแกรมเชิงวัตถุ ทั้ง encapsulation, composition, inheritance, polymorphism และ abstraction ในงานจริง
- ได้ฝึกอัลกอริทึมสำคัญ เช่น การค้นหาเส้นทางด้วย Dijkstra และ A*, การตรวจการชนแบบ swept collision และการคำนวณอันดับการแข่งขัน
- ได้ฝึกอ่านและตรวจสอบข้อมูลจากไฟล์ Tiled รวมถึงจัดการเส้นทางของไฟล์ภาพและข้อผิดพลาดของข้อมูล

ประโยชน์ต่อผู้เล่น

ผู้เล่นได้รับประสบการณ์การแข่งขันที่ต้องบริหาร stamina เลือกจังหวะเร่ง เก็บไอเทม หลบอุปสรรค และตัดสินใจว่าจะใช้เส้นทางหลักหรือเส้นทางลัด ผู้เล่นยังสามารถใช้การโจมตี การป้องกัน และการชาร์จพลังเพื่อสร้างความได้เปรียบหรือแก้ไขสถานการณ์ได้

ประโยชน์ในเชิงการศึกษา

โครงการนี้สามารถใช้เป็นตัวอย่างการเชื่อมโยงเนื้อหาหลายบทเข้าด้วยกัน ตั้งแต่การเขียนโปรแกรมเชิงวัตถุ โครงสร้างข้อมูล กราฟและการค้นหาเส้นทาง ไปจนถึงการออกแบบส่วนติดต่อผู้ใช้แบบ Event-driven และการจัดทำเอกสารประกอบโครงการ

1.4 ขอบเขตของโครงการ

ขอบเขตของโครงการกำหนดให้เกมทำงานบนแผนที่หลักหนึ่งแผนที่ คือ assets/Map/Background.tmx ซึ่งเป็นแผนที่แบบ Orthogonal ขนาด 900 × 200 tiles ขนาด tile 16 × 16 พิกเซล มีเลเยอร์ภาพหลายชั้น เลเยอร์ collision สำหรับพื้นที่ทึบ เลเยอร์ wet สำหรับพื้นที่น้ำ และเลเยอร์ race สำหรับจุดเกิด จุดตรวจ เส้นชัย และ waypoint ของบอท

ขอบเขตการทำงานที่พัฒนาเสร็จในโครงการมีดังนี้

- ผู้เล่นหนึ่งคนแข่งขันกับบอทห้าตัว โดยทุกตัวใช้กฎการเคลื่อนที่ stamina และการชนชุดเดียวกัน

- มีจุดตรวจสองจุดและเส้นชัยหนึ่งจุด ผู้เล่นและบอทต้องผ่านจุดตรวจตามลำดับก่อนเข้าเส้นชัย
- มีระบบ stamina และการเร่งความเร็วด้วยการกด Space สองครั้งภายในเวลาที่กำหนด
- มีไอเทม stamina, speed และ shield รวมถึงพื้นที่น้ำที่ลดความเร็ว
- มีระบบโจมตี ป้องกัน โล่ สตั๊น และสโลว์จากการต่อสู้
- มีระบบชาร์จพลังด้วยการกด O ค้าง เพื่อรับ stamina เมื่อชาร์จสำเร็จ
- มีบอทที่ใช้กราฟ waypoint การค้นหาเส้นทาง และบุคลิกภาพที่แตกต่างกัน
- มีหน้าจอ HUD แสดง stamina, สถานะ sprint, ความเร็วปัจจุบัน และข้อความแจ้งเหตุการณ์

สิ่งที่อยู่นอกขอบเขตของโครงการ ได้แก่ ระบบหลายแผนที่หรือหลายด่าน ระบบผู้เล่นหลายคนผ่านเครือข่าย ระบบบันทึกและโหลดเกม ระบบเสียง ระบบแก้ไขแผนที่ภายในเกม การรองรับอุปกรณ์เคลื่อนที่ และการฝังไฟล์ภาพทั้งหมดไว้ใน JAR โดยตรง

รายละเอียดขอบเขตและข้อเสนอโครงการฉบับเต็มให้แนบตามภาคผนวก ก ซึ่งเป็นส่วนที่นักศึกษาสามารถแทนที่ด้วย Proposal ฉบับจริงของรายวิชาได้

บทที่ 2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เรื่องราวของเกมเกิดขึ้นในฉากฤดูหนาวแบบแฟนตาซี ผู้เล่นรับบทเป็นนักขี่ม้าที่ต้องแข่งขันกับคู่แข่งห้าคนเพื่อไปให้ถึงเส้นชัยให้เร็วที่สุด เส้นทางแข่งขันมีทั้งเส้นทางหลักที่คดเคี้ยวและเส้นทางอ้อมที่ลึกกว่าเดิม ผู้เล่นต้องบริหาร stamina ให้เพียงพอต่อการเร่ง หลบสิ่งกีดขวาง เก็บไอเทม และตัดสินใจว่าจะเลือกเส้นทางใด

การเล่นเริ่มจากหน้าจอเมนู ผู้เล่นกดปุ่ม Marathon Race เพื่อเริ่มการแข่งขัน เกมจะนับถอยหลัง 3, 2, 1 แล้วเข้าสู่สถานะ RACING ผู้เล่นต้องวิ่งไปทางขวา ผ่านจุดตรวจทั้งสองจุดตามลำดับ และแตะเส้นชัย ระหว่างทางสามารถเก็บไอเทม stamina เพื่อเติมพลัง เก็บไอเทม speed เพื่อเพิ่มความเร็วชั่วคราว และเก็บไอเทม shield เพื่อป้องกันการโจมตีหรือการชน

การควบคุมหลักแสดงในตารางที่ 1

ปุ่ม	การทำงาน
Space	วิ่งไปทางขวา; กดซ้ำภายใน 0.3 วินาทีเพื่อเริ่ม sprint
W / S หรือ ↑ / ↓	ขยับขึ้น / ลง
O (กดค้าง)	ชาร์จพลังเพื่อรับ stamina เมื่อชาร์จครบตามเงื่อนไข
J	โจมตีคู่แข่งในระยะ
K (กดค้าง)	ป้องกันตัว ใช้ stamina ต่อเนื่อง
P หรือ Esc	หยุดชั่วคราว / กลับสู่เกม
F3	เปิด-ปิดกรอบตรวจสอบ collision และพื้นที่ต่าง ๆ

ลำดับสถานะของเกมคือ MENU ⟶ COUNTDOWN ⟶ RACING ⟶ FINISHED โดยในสถานะ COUNTDOWN ผู้เล่นยังเคลื่อนที่ไม่ได้ แต่สามารถกดปุ่มค้างไว้เพื่อให้เริ่มทำงานทันทีเมื่อถึงจังหวะ GO ระบบจะเก็บค่าปุ่มที่กดค้างไว้และไม่ล้างปุ่มที่ผู้เล่นยังกดอยู่เมื่อเริ่มการแข่งขัน

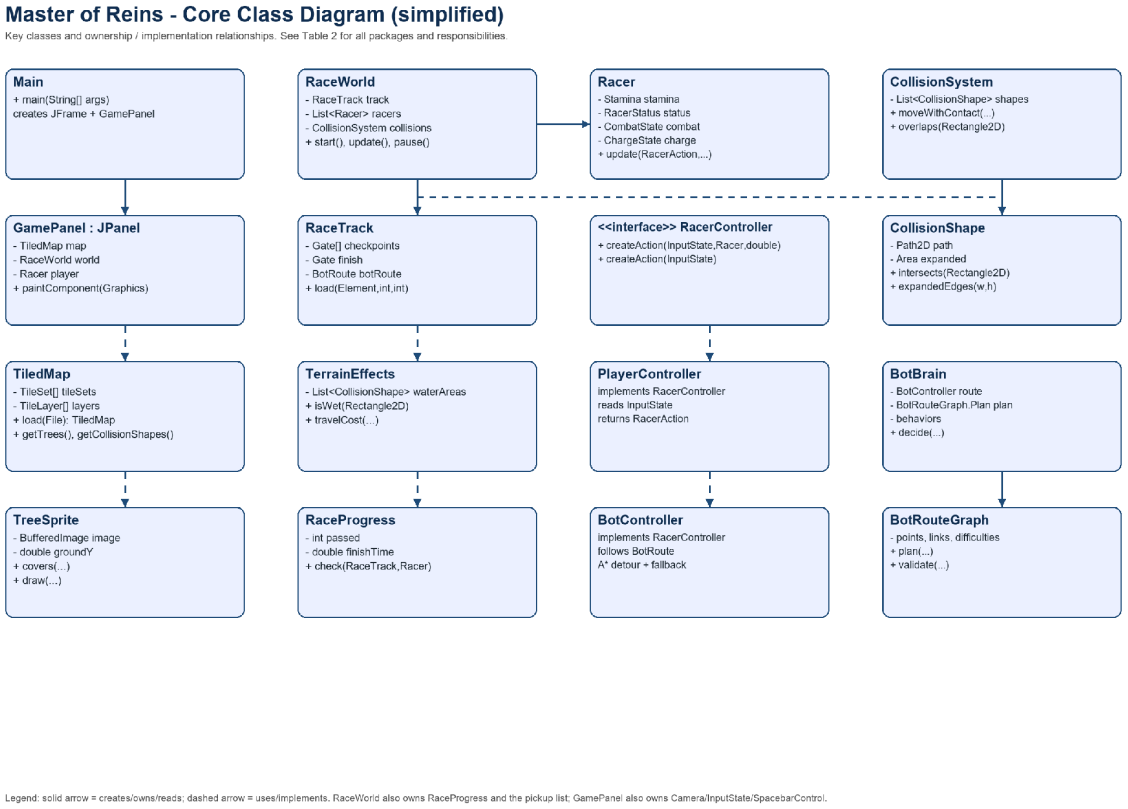
องค์ประกอบการเล่นที่สำคัญมีดังนี้

- Stamina ใช้สำหรับ sprint เมื่อ stamina หมด ผู้เล่นจะไม่สามารถ sprint ได้จนกว่าจะฟื้นหรือเก็บไอเทม
- การชนพื้นที่ทึบหรือขอบโลกขณะ sprint จะทำให้ติดสตั๊นและสโลว์ ส่วนการชนขณะวิ่งปกติจะทำให้สโลว์โดยไม่สตั๊น
- น้ำจะลดความเร็วลงเหลือ 80% และมีผลกับทั้งผู้เล่นและบอท

- ไอเทม stamina จะเกิดใหม่ตามเวลาที่กำหนด ส่วน speed และ shield มีระยะเวลาหรือจำนวนครั้งตามที่กำหนด

2.2 คลาสไดอะแกรมและคำอธิบาย

สถาปัตยกรรมของเกมแบ่งออกเป็นแพ็กเกจตามความรับผิดชอบ เพื่อลดการพึ่งพากันระหว่างส่วนภาพ ส่วนข้อมูลเกม และส่วนกฎกติกา แพ็กเกจ app ทำหน้าที่เป็นจุดเริ่มต้นและจัดการหน้าต่างเกม แพ็กเกจ world เก็บข้อมูลแผนที่และการแข่งขัน แพ็กเกจ racer เก็บสถานะและการเคลื่อนที่ของผู้แข่งขัน แพ็กเกจ collision ตรวจการชน แพ็กเกจ items เก็บไอเทมและแอนิเมชัน แพ็กเกจ combat จัดการการต่อสู้ และแพ็กเกจ bot จัดการ AI



รูปที่ 1 คลาสไดอะแกรมหลักของเกม (แบบย่อ)

รูปที่ 1 แสดงคลาสหลักและความสัมพันธ์แบบ owns/uses/implements โดย RaceWorld เป็นศูนย์กลางของสถานะการแข่งขัน มี RaceTrack, Racer, CollisionSystem, TerrainEffects, StaminaPickup และ BotBrain อยู่ภายใน Racer มี Stamina, RacerStatus, CombatState และ ChargeState เป็นองค์ประกอบย่อย ส่วน GamePanel ทำหน้าที่แสดงผลแผนที่ ผู้แข่งขัน ต้นไม้ และ HUD

ตารางที่ 2 สรุปหน้าที่ของคลาสสำคัญในแต่ละแพ็กเกจ

แพ็กเกจ	คลาสสำคัญ	หน้าที่หลัก
game.app	Main, GamePanel, Camera, InputState, SpacebarControl	เริ่มโปรแกรม แสดงหน้าจอ รับอินพุต จัดการกล้อง และวาด HUD
game.world	TiledMap, RaceWorld, RaceTrack, RaceProgress, TerrainEffects, TreeSprite	โหลดแผนที่และชน เก็บสถานะการแข่งขัน จุดตรวจ เส้นชัย น้ำ และต้นไม้
game.racer	Racer, RacerAction, RacerStatus, Stamina, ChargeState	เก็บสถานะผู้แข่งขัน การเคลื่อนที่ พลัง stamina สถานะสตั๊น/สโลว์ และการชาร์จ
game.collision	CollisionSystem, CollisionShape, CollisionMapLoader	อ่านพื้นที่ชนจาก Tiled ตรวจการชนแบบ swept และเลื่อนไปตามขอบ
game.items	StaminaPickup, SpriteAnimation, SweptTrigger	เก็บไอเทม แอนิเมชันการเกิดใหม่ และตรวจการเก็บไอเทม
game.combat	CombatSystem, CombatState, CombatAction, CombatRenderer	จัดการการโจมตี การป้องกัน โล่ ระยะเวลา และการแสดงผลการต่อสู้
game.bot	BotBrain, BotController, BotRouteGraph, BotNavigation, BotPersonality	สร้างเส้นทาง ค้นหาเส้นทาง เลือกพฤติกรรม และควบคุมบอท

RaceWorld ทำหน้าที่ประสานงานเกือบทั้งหมด เช่น เริ่มการแข่งขัน นับถอยหลัง อัปเดตผู้แข่งขันแต่ละตัว ตรวจการชน เก็บไอเทม ตรวจจุดตรวจ และจัดอันดับ RaceTrack เก็บจุดเกิด จุดตรวจ เส้นชัย และเส้นทางของบอท ส่วน RaceProgress แยกความคืบหน้าของผู้แข่งขันแต่ละคน เพื่อให้บอทและผู้เล่นมีสถานะการผ่านจุดตรวจของตนเอง

TiledMap อ่านไฟล์ TMX และ TSX โดยตรง ประกอบภาพกระเบื้องและต้นไม้จากไฟล์ PNG ภายนอก และเก็บข้อมูล collision จาก Object Layer ชื่อ collision ส่วน CollisionMapLoader แปลงรูปทรงที่รองรับ ได้แก่ รูปสี่เหลี่ยม รูปหลายเหลี่ยม และรูปวงรี ให้เป็น CollisionShape ในพิกัดโลก

2.3 รูปแบบการพัฒนา Application

เกมนี้เป็น Java Application แบบ Desktop ไม่ใช่ Applet และไม่ได้ทำงานบนเว็บเซิร์ฟเวอร์ โดยมี Main เป็นจุดเริ่มต้นในการสร้าง JFrame แล้วใส่ GamePanel เป็น content pane จากนั้น GamePanel จะโหลดแผนที่ สร้าง RaceWorld และเริ่ม Swing Timer สำหรับอัปเดตเกมทุกประมาณ 16 มิลลิวินาที

เทคโนโลยีที่ใช้มีดังนี้

- ภาษา Java ระดับ Java SE 17 ขึ้นไป ใช้ record, switch expression, text block และ pattern matching ตามความเหมาะสม
- Java Swing/AWT สำหรับหน้าต่าง ปุ่ม การวาดภาพสองมิติ และการจัดการ event
- Tiled Map Editor สำหรับสร้างแผนที่แบบ Orthogonal, ใช้ Tile Layer แบบ CSV และ Object Layer สำหรับ collision/race/wet
- ไฟล์ภาพ PNG และไฟล์ TSX/TMX ภายนอกที่โหลดผ่าน ImageIO และ Java XML DOM
- ไม่ใช้ไลบรารีเกมหรือไลบรารีกราฟิกภายนอก ทำให้โปรแกรมมี dependencies น้อยและสามารถรันด้วยไฟล์ JAR

โครงสร้างการทำงานแบ่งเป็นชั้น Presentation (GamePanel/Main), Domain (RaceWorld/TiledMap), Entity (Racer), System (Collision/Combat/Items) และ AI (BotBrain/BotRoute) การแบ่งชั้นนี้ทำให้การแก้ไขระบบหนึ่งไม่กระทบส่วนอื่นมากนัก เช่น การเปลี่ยนสูตรการชนทำได้ใน CollisionSystem โดยไม่ต้องแก้ GamePanel

การจัดส่งโปรแกรมทำได้สองรูปแบบ คือ รันจาก IDE ในโปรเจกต์ หรือสร้างไฟล์ JAR แล้วรันด้วยคำสั่ง java -jar MasterOfReins.jar โดยต้องวางโฟลเดอร์ assets และ asset-packs ไว้ในตำแหน่งเดียวกับ JAR เพราะโค้ดโหลดภาพและแผนที่จากไฟล์ภายนอก ไม่ได้ฝังไว้ใน classpath

2.4 การประยุกต์ใช้แนวคิด OOP

2.4.1 Constructor

Constructor ถูกใช้เพื่อกำหนดสถานะเริ่มต้นของวัตถุและตรวจสอบข้อมูลที่จำเป็น ตัวอย่างเช่น Racer รับตำแหน่ง ขนาด และ Stamina ใน constructor เพื่อให้ทุกตัวมีพลังและกรอบการเคลื่อนที่ครบก่อนเริ่มเกม

```
public Racer(int x, int y, int width, int height, Stamina stamina) {
    this.x = x; this.y = y;
    this.width = width; this.height = height;
    this.stamina = stamina;
}
```

ตัวอย่าง constructor ของ Racer

GamePanel มี constructor ที่โหลดแผนที่ สร้าง RaceWorld โหลดภาพตัวละครและแอนิเมชัน แล้วตั้งค่า Timer สำหรับเกมลูป หากโหลดแผนที่ไม่ได้ จะเก็บข้อผิดพลาดไว้และแสดงข้อความแทนการ crash ทั้งโปรแกรม ส่วน RaceWorld constructor ตรวจสอบว่า track ไม่เป็น null และสร้างผู้เล่น บอท และรายการไอเทมก่อนเริ่มเกม

นอกจาก constructor แบบปกติแล้ว โปรเจกต์ยังใช้ compact constructor ของ record ในการตรวจสอบค่า เช่น StaminaPickup.Definition ตรวจว่า image, type, opacity, respawnSeconds และค่าพลังงานมีความถูกต้อง หากค่าไม่ถูกต้องจะโยน IllegalArgumentException ทันที ทำให้ข้อผิดพลาดถูกพบตั้งแต่ตอนโหลดข้อมูล

2.4.2 Encapsulation

Encapsulation คือการซ่อนสถานะภายในและเปิดเฉพาะ method ที่จำเป็น โปรเจกต์ใช้ private field เป็นหลัก เช่น Racer เก็บ velocityX, velocityY, stunRemaining และ slowRemaining ไว้ภายใน ไม่ให้คลาสอื่นแก้ไขโดยตรง คลาสอื่นอ่านค่าผ่าน getter เช่น getX(), getY(), getFeetBounds(), isStunned() และ getSpeed()

CollisionShape เก็บ Path2D, Area ที่ขยายแล้ว และรายการ edge ไว้ภายใน เพื่อให้การตรวจการชนใช้ข้อมูลที่คำนวณไว้ซ้ำได้อย่างมีประสิทธิภาพ ส่วน RaceProgress เก็บจำนวนจุดตรวจที่ผ่านและเวลาเข้าเส้นชัย โดยเปิด method check() และ getPassed() ให้ RaceWorld เรียกใช้

2.4.3 Composition

Composition ปรากฏชัดในความสัมพันธ์แบบ “มีอยู่ภายใน” ตัวอย่างเช่น GamePanel มี TiledMap, RaceWorld, Camera, InputState และ SpacebarControl เป็นองค์ประกอบ RaceWorld มี RaceTrack, รายการ Racer, CollisionSystem, TerrainEffects, รายการ StaminaPickup, CombatSystem และ BotBrain

Racer ประกอบด้วย Stamina, RacerStatus, CombatState และ ChargeState ซึ่งแยกความรับผิดชอบด้านพลัง สถานะการต่อสู้ และการชาร์จออกจากกัน TiledMap ประกอบด้วย TileSet, TileLayer และ TreeSprite หลายตัว ส่วน BotBrain ประกอบด้วย BotController, BotRouteGraph.Plan และพฤติกรรมต่าง ๆ เช่น BotItemBehavior, BotPassingBehavior และ BotChargeBehavior

2.4.4 Polymorphism

Polymorphism ถูกใช้ผ่าน interface RacerController ซึ่งมี method createAction(InputState, Racer, double) ให้ PlayerController และ BotController implement ความแตกต่างคือ PlayerController อ่านค่าจาก InputState ส่วน BotController อ่านเส้นทางและตัดสินใจทิศทางเอง RaceWorld เรียกใช้ผ่านตัวแปรประเภท RacerController โดยไม่ต้องรู้ว่าเป็นผู้เล่นหรือบอท

```
public interface RacerController {
    RacerAction createAction(InputState input, Racer racer, double
seconds);
}
// RaceWorld เรียกใช้ผ่าน interface
RacerAction action = controller.createAction(input, racer, step);
```

ตัวอย่าง polymorphism ผ่าน RacerController

นอกจากนี้ GamePanel ยังมี interface ภายใน Renderable ซึ่งมี RacerRenderable และ TreeRenderable เป็น implementation ทำให้สามารถเรียงลำดับการวาดผู้แข่งขันและต้นไม้ตามค่า groundY ได้ด้วยคอลเลกชันเดียว

Polymorphism ยังเกิดจากการ override method ของ Swing เช่น RoundedButton, PauseButton และ RoundedPanel override paintComponent() เพื่อวาดปุ่มและ panel แบบกำหนดเอง ขณะที่ GamePanel override paintComponent() เพื่อวาดฉากเกมทั้งหมด

2.4.5 Abstraction

โปรเจกต์ใช้ Abstraction ผ่าน interface เป็นหลัก ได้แก่ RacerController และ Renderable ซึ่งกำหนดสัญญาของพฤติกรรมโดยไม่ผูกกับรายละเอียดการทำงาน ผู้เรียกใช้รู้เพียงว่าต้องส่ง InputState, Racer และเวลาเข้าไป แล้วจะได้ RacerAction กลับมา จึงสามารถเปลี่ยนหรือเพิ่ม controller ได้โดยไม่ต้องแก้ RaceWorld

2.4.6 Inheritance

Inheritance ถูกใช้กับส่วนติดต่อผู้ใช้ของ Swing อย่างชัดเจน GamePanel extends JPanel เพื่อให้เป็นหน้าที่วาดฉากเกมได้ RoundedButton extends JButton เพื่อเพิ่มรูปทรงปุ่มแบบมน PauseButton extends RoundedButton เพื่อเพิ่มไอคอนหยุดชั่วคราว และ RoundedPanel extends JPanel เพื่อวาดพื้นหลังแบบมน

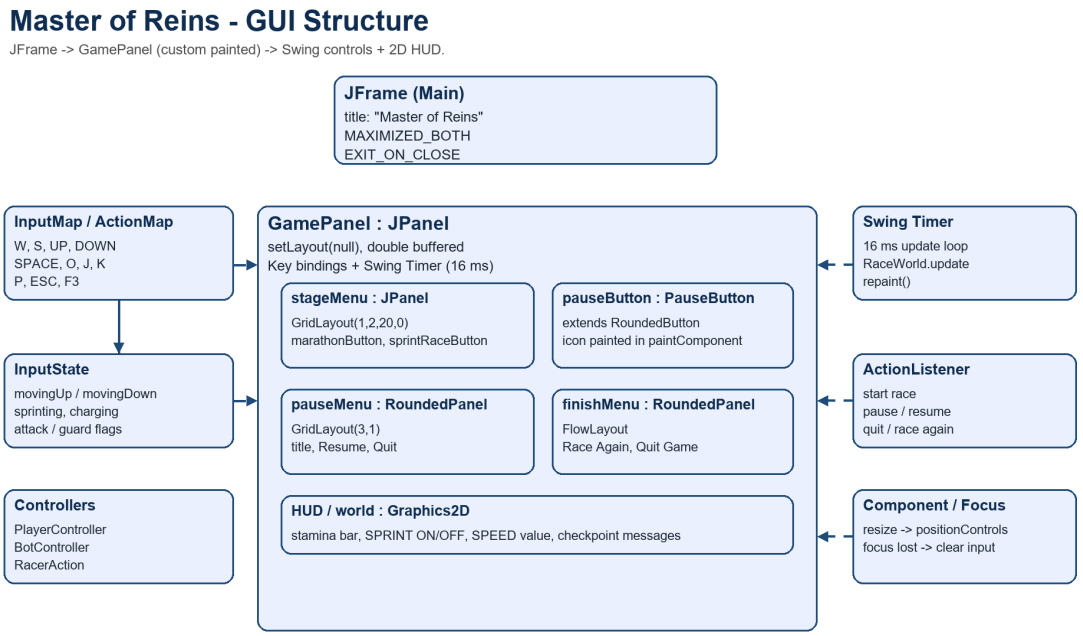
```
public final class GamePanel extends JPanel { ... }
private static class RoundedButton extends JButton { ... }
private static final class PauseButton extends RoundedButton { ... }
```

ตัวอย่าง inheritance ในส่วน GUI

คลาสลูกสามารถเรียก super.paintComponent() เพื่อให้ Swing วาดองค์ประกอบพื้นฐานก่อน แล้วจึงวาดรายละเอียดเพิ่มเติม การสืบทอดช่วยลดโค้ดซ้ำและทำให้พฤติกรรมร่วมของปุ่มถูกกำหนดไว้ที่คลาสฐาน

2.5 หน้าจอ GUI และส่วนประกอบ

หน้าจอเกมเป็นหน้าต่าง Java Swing ขนาดเต็มจอ (MAXIMIZED_BOTH) ภายในมี GamePanel ทำหน้าที่วาดฉากเกมด้วย Graphics2D และมี Swing component สำหรับปุ่มและเมนูต่าง ๆ โครงสร้าง GUI แสดงในรูปที่ 2



รูปที่ 2 โครงสร้าง GUI และเส้นทางการเกิด event

ส่วนประกอบหลักของ GUI มีดังนี้

Component	ประเภท	หน้าที่
JFrame	Swing window	หน้าต่างหลัก ตั้งชื่อเกมและปิดโปรแกรมเมื่อปิดหน้าต่าง
GamePanel	JPanel	วาดแผนที่ ผู้เล่น บอท ต้นไม้ HUD และจัดการ input
stageMenu	JPanel (GridLayout)	เมนูเริ่มเกม ประกอบด้วย Marathon Race และ Flat Sprint Race
marathonButton	RoundedButton	เริ่มการแข่งขันโหมด marathon
sprintRaceButton	RoundedButton	ปุ่มโหมด Flat Sprint (ยังไม่เปิดใช้ในแผนที่นี้)
pauseButton	PauseButton	ปุ่มหยุดชั่วคราวที่มุมขวาบน
pauseMenu	RoundedPanel (GridLayout)	เมนู PAUSED พร้อม Resume และ Quit Game
finishMenu	RoundedPanel (FlowLayout)	เมนูหลังจบเกม พร้อม Race Again และ Quit Game

Component	ประเภท	หน้าที่
HUD	Graphics2D	แสดง stamina, SPRINT ON/OFF, SPEED และข้อความแจ้งเหตุการณ์

ตารางที่ 3 สรุปส่วนประกอบหลักของ GUI และหน้าที่ของแต่ละองค์ประกอบ

HUD ถูกวาดด้วย Graphics2D ไม่ได้ใช้ JLabel ทำให้ควบคุมตำแหน่ง ขนาด และสีได้อิสระ โดยแสดงแถบ stamina ข้อความ SPRINT ON/OFF และค่าความเร็วปัจจุบันเป็นตัวเลขบน panel มุมขวาล่าง



รูปที่ 3 หน้าจอการแข่งขันและ HUD มุมขวาล่าง

2.6 Event Handling

เกมใช้ event handling สองกลุ่มหลัก คือ กลุ่มคีย์บอร์ดผ่าน InputMap/ActionMap ของ Swing และกลุ่มปุ่ม/หน้าต่างผ่าน ActionListener, ComponentListener และ FocusListener

การผูกคีย์ใช้ WHEN_IN_FOCUSED_WINDOW ทำให้เกมยังรับคีย์แม้โฟกัสอยู่ใน component ลูก ตารางที่ 4 สรุป event ที่สำคัญ

Event / Component	การทำงาน
Key pressed/released: W, S, UP, DOWN	ปรับ movingUp/movingDown ใน InputState
Key pressed/released: SPACE	เริ่ม/หยุดวิ่งไปทางขวา และตรวจการกดซ้ำเพื่อ sprint ผ่าน SpacebarControl

Event / Component	การทำงาน
Key pressed/released: O	เริ่ม/หยุดสถานะ charging สำหรับการชาร์จพลัง
Key pressed: P หรือ Esc	หยุดชั่วคราวหรือกลับสู่เกม
Key released: F3	สลับการแสดงกรอบ collision
ActionListener ของปุ่ม	เริ่มแข่ง หยุดชั่วคราว กลับเมนู หรือแข่งใหม่
Swing Timer 16 ms	เรียก updateGame() เพื่ออัปเดต RaceWorld และ repaint
ComponentListener	จัดตำแหน่งปุ่มใหม่เมื่อหน้าต่างเปลี่ยนขนาด

SpacebarControl แยกการกด Space ครั้งใหม่จาก key repeat และตรวจการกดซ้ำภายใน 300 มิลลิวินาทีเพื่อเริ่ม sprint ทำให้การกดค้างหนึ่งครั้งเป็นการวิ่งปกติ และการกดซ้ำเป็นการเร่งความเร็ว

สำหรับการเริ่มการแข่งขันใหม่ ระบบจะเก็บปุ่มที่ผู้เล่นกดค้างไว้ เช่น S และ O เพื่อให้เริ่มทำงานต่อเนื่องเมื่อถึง GO แต่จะล้างเฉพาะ action ที่เป็น one-shot เช่น attack request เพื่อไม่ให้เกิดการโจมตีค้างจากเมนู

2.7 อัลกอริทึมที่สำคัญ

2.7.1 Game Loop และลำดับการอัปเดต

RaceWorld.update() รับเวลาเป็นวินาทีและแบ่งการอัปเดตเป็นขั้นละไม่เกิน 1/60 วินาที เพื่อให้การเคลื่อนที่และการชนมีเสถียรภาพแม้เฟรมเรตของหน้าจอไม่คงที่ ในแต่ละขั้นจะอัปเดตไอเทม ตรวจสถานะผู้แข่งขัน สร้าง action จาก controller ตรวจการต่อสู้ เคลื่อนที่ผู้แข่งขัน ตรวจการชาร์จ เก็บไอเทม ตรวจจุดตรวจ/เส้นชัย และเพิ่มเวลาแข่งขัน

2.7.2 Collision Detection

การชนใช้กรอบเท้าของผู้แข่งขัน (Feet Bounds) แทนกรอบภาพทั้งตัว เพื่อให้การชนดูสมจริงและไม่ติดภาพส่วนหัวหรือหางม้า กรอบเท้าถูกกวาดไปตามเส้นทางการเคลื่อนที่ของเฟรม (Swept Collision) แล้วตรวจกับพื้นที่ชนที่ถูกขยายด้วยขนาดครึ่งหนึ่งของกรอบเท้า (Minkowski expansion) ทำให้จุดศูนย์กลางเท้าเคลื่อนที่ไปชนขอบที่ขยายแทนการตรวจกรอบซ้อนกรอบ

CollisionSystem.moveWithContact() วนแก้ไขการชนได้หลายรอบ เพื่อให้ผู้แข่งขันเลื่อนไปตามขอบกำแพง (Slide) แทนการหยุดทันที และคืนค่า MotionSegment สำหรับใช้ตรวจจุดตรวจ/เส้นชัยแบบ swept เช่นเดียวกับการชน

```
Movement m = collisions.moveWithContact(feet, dx, dy, centreLimits);
x += m.dx(); y += m.dy();
// m.hit() == true แปลว่ามีการชน และ m.path() เก็บช่วงการเคลื่อนที่
```

2.7.3 จุดตรวจและอันดับ

RaceTrack เก็บ Gate ของจุดตรวจและเส้นชัยเป็น Rectangle2D และใช้ SweptTrigger.entryFraction() เพื่อหาว่าการเคลื่อนที่ของเท้าเข้าสู่กรอบหรือไม่ แม้กรอบจะบางมากก็ไม่ถูกข้ามไป RaceProgress ตรวจจุดตรวจตามลำดับโดยใช้ passed เป็นตัวชี้ จึงป้องกันการข้ามจุดตรวจหรือการนับจุดตรวจผิดลำดับ

2.7.4 Terrain Effects และน้ำ

TerrainEffects เก็บพื้นที่น้ำเป็น CollisionShape ที่ไม่แข็ง เมื่อเท้าอยู่ในพื้นที่น้ำ ความเร็วจะถูกคูณด้วย WATER_SPEED_MULTIPLIER (0.80) และใช้ travelCost() ในการประเมินเส้นทางของบอท โดยคิดค่าน้ำเป็นระยะทางที่แพงขึ้นตามความ aversion ของบุคลิกภาพ

2.7.5 AI ของบอท

บอทใช้ BotRouteGraph ซึ่งเป็นกราฟมีทิศทาง จุดแต่ละจุดมี order, next และ nextDifficulty สถานะในการค้นหาประกอบด้วย (waypoint, จำนวนจุดตรวจที่ผ่าน, finished) และใช้ Dijkstra เพื่อหาเส้นทางที่มีต้นทุนต่ำสุด โดยต้นทุนประกอบด้วยระยะทาง ค่าน้ำ ค่าความยากของเส้นเชื่อม และอิทธิพลจากบอทอื่น

ที่ทางแยก บอทจะเลือกเส้นทางจากกราฟ เช่น เส้นทางหลักหรือเส้นทางอ้อมที่ลึกกว่า บอทสองตัวถูกกำหนดให้เลือกเส้นทางอ้อมเพื่อให้การแข่งขันมีความหลากหลาย ส่วนบอทตัวอื่นเลือกเส้นทางหลัก

BotController แปลงจุดหมายเป็นคำสั่งแปดทิศ (ซ้าย/ขวา/ขึ้น/ลง/ทแยง) และใช้ BotNavigation ซึ่งเป็น A* บนกริดขนาด 16 พิกเซลเพื่อหาเส้นทางอ้อมระยะสั้นเมื่อมีสิ่งกีดขวางหรือน้ำ หากหาไม่พบจะใช้ fallbackMove() เพื่อลองก้าวข้ามหรือหลบด้านข้างแทนการหยุดนิ่ง

2.7.6 แอนิเมชันและการเรียงลำดับภาพ

TiledMap อ่านแอนิเมชันของ tile จาก TSX และใช้เวลา animationSeconds ในการเลือกเฟรมตาม duration ของแต่ละ frame ส่วน SpriteAnimation ใช้กับภาพผู้แข่งขันและแอนิเมชันวิ่ง โดยเปลี่ยนเฟรมตามเวลาและสถานะ moving/sprinting

2.7.7 การต่อสู้และการชาร์จ

CombatSystem ตรวจระยะการโจมตี มุมหรือทิศทาง และสถานะการป้องกันของผู้ถูกโจมตี หากโจมตีถูกจะเกิดสตั๊น/สโลว์หรือโล่ป้องกัน ขึ้นอยู่กับสถานะ CombatState และ RacerStatus ส่วน ChargeState ตรวจเวลากด O ค้าง หากชาร์จครบ 3–5 วินาทีจะได้ stamina ตามเวลา และหากชนสิ่งกีดขวางหรือผ่านน้ำระหว่างชาร์จจะถือว่าล้มเหลวและติดสตั๊น เว้นแต่มีโล่ป้องกัน

บทที่ 3 สรุป

3.1 ปัญหาที่พบระหว่างการพัฒนา

ระหว่างการพัฒนาเกมพบปัญหาหลายลักษณะ ตั้งแต่ข้อมูลแผนที่ ระบบชน ระบบบอท ไปจนถึงส่วนติดต่อผู้ใช้ ตารางที่ 5 สรุปปัญหาสำคัญและแนวทางแก้ไข

ปัญหา	สาเหตุ	แนวทางแก้ไข
โหลดแผนที่แล้วได้ค่า null	ค่า treeBaseX/treeBaseY ของ tileset ต้นไม้อยู่นอกขนาดภาพ ทำให้ TiledMap โหลดไม่สำเร็จ	ปรับค่าจุดฐานให้อยู่ในขอบเขตภาพและตรวจสอบขนาด tileset ก่อนโหลด
บอทหยุดนิ่งหรือวนโจมตี/ป้องกัน	การวางแผนเส้นทางที่จุดแยกคืนค่าว่าง ทำให้ controller ไม่มีเป้าหมายและหยุด	แก้จุดเริ่มต้นของ planner ตรวจสอบ plan ซ้ำ และมี fallback movement
บอทเลือกเส้นทางอ้อมไม่ได้	กราฟ waypoint ไม่มีเส้นเชื่อมไปยังจุดเริ่มเส้นทางอ้อม	เพิ่ม next ของ waypoint ทางแยกและกำหนดบอทสองตัวให้เลือกเส้นทางอ้อม
กรอบไอเทมเปลี่ยนตามแอนิเมชัน	getArea() คำนวณจากพิกเซลที่มองเห็นของเฟรมปัจจุบัน	กำหนด hitbox คงที่จากขนาดภาพไอเทมและใช้ hitbox เดียวตลอดแอนิเมชัน
ไฟล์ภาพมีขนาดใหญ่และอยู่ภายนอก JAR	asset-packs มีไฟล์ต้นฉบับจำนวนมาก และโค้ดโหลดจาก File	จัดส่ง JAR พร้อม assets/asset-packs หรือบีบอัดเป็นไฟล์ ZIP

ปัญหาที่สำคัญที่สุดคือการทำงานร่วมกันระหว่างข้อมูลจาก Tiled กับโค้ด Java เนื่องจาก Tiled เก็บพิกัดและขนาดภาพในหน่วยพิกเซล แต่โค้ดต้องแปลงเป็นตำแหน่ง tile และ world coordinate หลายชั้น ทำให้ข้อผิดพลาดเล็กน้อย เช่น ค่าจุดฐานต้นไม้เกินภาพ ส่งผลให้โหลดแผนที่ทั้งแผนที่ไม่ได้

ข้อจำกัดที่ยังเหลืออยู่ ได้แก่ เกมมีแผนที่เดียว ไม่มีระบบเสียง ไม่มีระบบบันทึกเกม และไฟล์ภาพไม่ได้ถูกฝังใน JAR ทำให้การส่งต่อต้องระวังเรื่อง path ของ assets และ asset-packs

3.2 จุดเด่นของโปรแกรม

จุดเด่นของ Master of Reins คือการเป็นเกมที่พัฒนาด้วย Java พื้นฐานโดยไม่ใช้ game engine แต่มีองค์ประกอบครบทั้งการแข่งขัน การต่อสู้ ไอเทม บอท และส่วนติดต่อผู้ใช้ที่ใช้งานได้จริง

- ระบบแผนที่จาก Tiled ที่รองรับหลายเลเยอร์ หลาย tileset ภาพต้นไม้ และ object layer สำหรับ collision/race/wet

- ระบบชนแบบ swept collision ที่รองรับรูปหลายเหลี่ยม วงรี และการเลื่อนไปตามขอบ ทำให้การเคลื่อนที่ลื่นและไม่ทะลุกำแพงบาง
- ระบบเรียงลำดับต้นไม้ตามความลึก (Y-sorting) และการลดความทึบของต้นไม้เมื่อบังผู้เล่น
- บอทที่ใช้กราฟ waypoint, Dijkstra, A* ระยะสั้น และบุคลิกภาพที่แตกต่างกัน ทำให้การแข่งขันมีรูปแบบหลากหลาย
- ระบบ stamina, sprint, ไอเทม, โล่, การโจมตี, การป้องกัน และการชาร์จพลังที่ทำงานร่วมกัน
- โค้ดแบ่งแพ็กเกจตามความรับผิดชอบและใช้ OOP อย่างสม่ำเสมอ ทำให้อ่านและต่อยอดได้ง่าย

โดยรวม โครงการนี้แสดงให้เห็นว่าการนำองค์ความรู้ด้านการเขียนโปรแกรมเชิงวัตถุ อัลกอริทึม และการออกแบบส่วนติดต่อผู้ใช้มาประกอบกันเป็นเกมที่เล่นได้จริงเป็นไปได้ แม้จะอยู่ในขอบเขตของรายวิชาและใช้เวลาพัฒนาจำกัด

คำแนะนำสำหรับผู้สอนที่อยากให้อธิบายหรือที่เรียนแล้วไม่เข้าใจหรืออยากให้เพิ่มสำหรับน้อง ๆ รุ่นต่อไป

อาจารย์อธิบายเข้าใจมากค่ะแต่บางทีกระดานฝั่งที่อาจารย์เขียนมันสะท้อนแสงเป็นบางครั้ง ทำให้บางครั้งก็ไม่เห็นเนื้อความที่อาจารย์อธิบายไปบ้าง

เอกสารอ้างอิง

Oracle. (2025). Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/

Tiled. (2025). Tiled Documentation: TMX Map Format and TSX Tileset Format. https://doc.mapeditor.org/

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics.

Red Blob Games. (2024). Introduction to A* and Dijkstra pathfinding. https://www.redblobgames.com/pathfinding/

GandalfHardcore. (2024). GandalfHardcore FREE Platformer Assets. (สื่อประกอบเกม)

REalOne. (2024). Pixel Crawler - Free Pack. (สื่อประกอบเกม)

ภาคผนวก ก แนวทางการติดตั้งและรันโปรแกรม

ภาคผนวกนี้ใช้เป็นคู่มือสำหรับอาจารย์หรือผู้ตรวจโครงการในการเปิดเกม และเป็นที่สำหรับแนบ Proposal ฉบับเต็มตามข้อกำหนดของรายวิชา

ก.1 สิ่งที่ต้องเตรียม

- Java Runtime Environment (JRE) หรือ JDK เวอร์ชัน 17 ขึ้นไป
- ไฟล์ MasterOfReins.jar
- โฟลเดอร์ assets และ asset-packs ที่อยู่ระดับเดียวกับไฟล์ JAR

ก.2 วิธีการรัน

เปิด Command Prompt หรือ PowerShell ที่โฟลเดอร์ซึ่งมีไฟล์ JAR แล้วพิมพ์

```
java -jar MasterOfReins.jar
```

คำสั่งรันเกม

หากเกมแสดงข้อความว่าโหลดแผนที่ไม่ได้ ให้ตรวจสอบว่าโฟลเดอร์ assets/Map มี Background.tmx และไฟล์ TSX ครบ รวมถึงโฟลเดอร์ asset-packs อยู่ถูกตำแหน่ง